

Proyecto Nuclear - Plataforma de Empleos

Docente: Rubén Darío Díaz Ruíz

**CUE Alexander Von Humboldt
Facultad de Ingeniería**

**Jan Carlos Guevara
Andrea Jaramillo Gutierrez**

Armenia - Quindio

2026

1. INTRODUCCIÓN

El Proyecto Nuclear IV plantea el diseño de una plataforma institucional de empleabilidad y prácticas profesionales desarrollada desde su concepción bajo una arquitectura de microservicios y desplegada en un entorno cloud altamente escalable.

El sistema está pensado para resolver una problemática clave en Colombia: la desarticulación entre estudiantes universitarios, empresas y unidades académicas responsables de la gestión de prácticas y empleabilidad. Las herramientas existentes suelen ser genéricas, no se alinean con los ciclos formativos, no incluyen gestión documental especializada y carecen de indicadores institucionales útiles para la toma de decisiones académicas y la acreditación de programas.

La plataforma propuesta atiende esta realidad mediante un ecosistema distribuido compuesto por servicios independientes, seguros y altamente disponibles. Cada microservicio atiende una área funcional específica del negocio: perfiles, vacantes, emparejamiento inteligente, postulaciones, prácticas profesionales, analítica institucional, notificaciones, almacenamiento documental y auditoría.

El modelo permite evolucionar los componentes del sistema de forma autónoma, escalar servicios críticos según la demanda, incorporar algoritmos avanzados de recomendación y garantizar continuidad operativa en momentos de alto uso como convocatorias, postulaciones masivas o procesos de evaluación de prácticas.

Este enfoque garantiza una solución moderna, sostenible y alineada con estándares internacionales de calidad como ISO/IEC 25010, aplicando además buenas prácticas de seguridad, accesibilidad, ética algorítmica y cumplimiento normativo colombiano.

2. EJE PROBLÉMICO Y JUSTIFICACIÓN

La empleabilidad juvenil en Colombia presenta desafíos significativos derivados de:

- La falta de una plataforma especializada para estudiantes universitarios.
- La ausencia de herramientas estructuradas para el seguimiento de prácticas profesionales.
- Procesos de selección lentos y manuales entre universidades y empresas.



- Falta de indicadores de empleabilidad que respalden decisiones institucionales.
- Dificultades para conectar competencias del estudiante con las necesidades reales del sector productivo.

Lo anterior limita la inserción laboral oportuna, obstaculiza la gestión institucional y reduce la efectividad de los procesos de formación. La plataforma basada en microservicios surge como respuesta directa a estos desafíos, permitiendo modularidad, alta disponibilidad, integración ágil y análisis continuo del comportamiento de los procesos.

La elección de una arquitectura de microservicios desde el inicio se fundamenta en:

Necesidades del dominio

El ecosistema de empleabilidad requiere módulos completamente independientes:

- Perfiles
- Vacantes
- Matching
- Postulaciones
- Prácticas
- Analítica
- Notificaciones

Requerimientos de escalabilidad

Los picos de uso (convocatorias, postulaciones masivas) afectan sólo ciertos servicios. Escalar por separado reduce costos y aumenta la capacidad del sistema.

Evolución constante

Cada microservicio puede actualizarse o reemplazarse sin interrumpir el resto del sistema.

Diversidad tecnológica

Permite usar distintos lenguajes optimizados según la necesidad:

- Node.js: Alta concurrencia
- Java Spring Boot: Transacciones críticas
- Python: Algoritmos de matching y analítica

Cumplimiento institucional y normativo

Microservicios permiten:

- Seguridad granular
- Bases de datos aisladas
- Auditoría detallada por servicio

3. OBJETIVO GENERAL

Diseñar la arquitectura integral de una plataforma distribuida para la gestión de vacantes, perfiles estudiantiles, emparejamiento inteligente, postulaciones y prácticas profesionales, implementada completamente mediante microservicios independientes y desplegada en la nube, garantizando escalabilidad, seguridad, disponibilidad continua y cumplimiento del estándar internacional ISO/IEC 25010.

¿Por qué este objetivo?

- Microservicios: Porque el dominio del sistema está claramente dividido en submódulos independientes: vacantes, usuarios, matching, postulaciones, prácticas, analítica.
- Despliegue en la nube: Porque la demanda es variable y se requiere elasticidad real.
- ISO/IEC 25010: Porque la universidad necesita demostrar calidad de software y confiabilidad.
- Alta disponibilidad: Porque estudiantes y empresas deben usar el sistema permanentemente.
- Emparejamiento inteligente: Porque mejora la eficiencia y relevancia de las postulaciones.

4. ANÁLISIS DE REQUISITOS Y CASOS DE USO

4.1 Requisitos Funcionales

- Gestión de Perfiles

RF01: Crear perfiles de usuario (estudiante, empresa, administrador).

RF02: Actualizar información académica, certificaciones y documentos.

RF03: Validar identidad de empresas mediante analistas internos.

- Vacantes

RF04: Publicar vacantes con filtros especializados (programa, semestre, modalidad, salario).

RF05: Control de versiones y estados de vacantes (borrador, revisión, aprobada, cerrada).

RF06: Buscador avanzado basado en ElasticSearch.

- Matching Inteligente

RF07: Ejecutar algoritmo de emparejamiento basado en compatibilidad perfil-vacante.

RF08: Generar recomendación automática diaria para cada estudiante.

- Postulación

RF09: Permitir subir documentos: hoja de vida, carta, certificaciones.

RF10: Seguimiento del estado de la postulación.

RF11: Notificaciones automáticas (correo/SMS/WhatsApp).

- Prácticas Profesionales

RF12: Registro de convenio, tutor empresarial y tutor académico.

RF13: Registro de informes parciales y evaluaciones finales.

RF14: Emitir certificado de cumplimiento.

- Analíticas

RF15: Mostrar métricas por programa, facultad y sede.

RF16: Exportar informes en PDF/Excel.



- Seguridad

RF17: Autenticación con JWT/OAuth2.

RF18: Manejo seguro de roles (RBAC).

- Operación

RF19: Logs de auditoría por cada acción crítica.

RF20: Integración con la nube para archivos (S3).

4.2 Requisitos No Funcionales

- Rendimiento

RNF01: Tiempo de respuesta < 1.5 segundos.

RNF02: El sistema debe soportar 300–500 solicitudes por segundo.

- Seguridad

RNF03: Cifrado TLS 1.3.

RNF04: Cumplimiento Ley 1581 Habeas Data.

RNF05: Sanitización de inputs (OWASP).

- Disponibilidad

RNF06: Alta disponibilidad $\geq 99.9\%$.

RNF07: Recuperación automática ante fallos (self-healing Kubernetes).

- Escalabilidad

RNF08: Escalado horizontal automático por CPU y tráfico.

- Mantenibilidad

RNF09: Código modular basado enDDD.

RNF10: Testing $\geq 70\%$ de cobertura.}



- Calidad (ISO 25010)

RNF11: Usabilidad compatible con WCAG 2.1.

RNF12: Portabilidad entre entornos (Dockerized).

4.3 Casos de Uso

CU	DESCRIPCIÓN	RF
CU01	Registro de usuario	RF01, RF017
CU02	Publicación de vacante	RF04, RF05
CU03	Postulación	RF09, RF10
CU04	Matching inteligente	RF07
CU05	Notificaciones	RF11
CU06	Crear convenio de práctica	RF12
CU07	Evaluación final	RF13
CU08	Descarga de métricas	RF16

4.4 Casos de Uso

CU01 – Registro de Usuario

- Actor: Estudiante/Empresa
- RF Asociado: RF01, RF17
- Precondición: No debe existir un usuario registrado con el mismo correo electrónico.
- Flujo principal:
 1. El actor accede a la pantalla de registro.
 2. Ingresar datos obligatorios (nombre, correo, contraseña, rol).
 3. El microservicio de Usuarios valida la información.
 4. El microservicio de Autenticación crea las credenciales y genera el token inicial.
 5. El sistema registra al usuario en su base de datos correspondiente.

6. El actor inicia sesión en la plataforma.
- **Postcondición:** El usuario queda registrado con credenciales activas y su perfil inicial disponible.

CU02 – Publicar Vacante

- Actor: Empresa
- RF Asociado: RF04, RF05
- Flujo principal:
 1. La empresa accede al módulo de vacantes.
 2. Ingresa los datos de la oferta: título, descripción, habilidades requeridas, modalidad, salario, etc.
 3. El microservicio de Vacantes valida formato, campos y permisos del actor.
 4. El backend registra la vacante en la base de datos del microservicio.
 5. Se indexa la vacante para permitir búsquedas rápidas.
 6. La vacante queda visible para estudiantes.
- **Postcondición:** La vacante queda registrada y publicada correctamente.

CU03 – Postularse a Vacante

- Actor: Estudiante
- RF Asociado: RF09, RF10
- **Precondición:** Debe existir una vacante activa
- Flujo principal:
 1. El estudiante selecciona una vacante publicada.
 2. Adjunta documentos requeridos (hoja de vida, carta de presentación, anexos).
 3. El microservicio de Archivos almacena los documentos.
 4. El microservicio de Postulaciones valida el estado del estudiante y la vacante.
 5. Se registra la postulación en la base de datos.
 6. El sistema genera una notificación al estudiante y a la empresa.
- **Postcondición:** Postulación registrada y asociada correctamente a una vacante.

CU04 – Matching Inteligente

- Actor: Estudiante



- RF Asociado: RF07
- Flujo principal:
 1. El sistema identifica el perfil del estudiante (academia, competencias, historial).
 2. El microservicio de Matching obtiene las vacantes disponibles.
 3. Se ejecuta el algoritmo de emparejamiento perfil–vacante.
 4. El sistema calcula un ranking de compatibilidad.
 5. Se muestran recomendaciones personalizadas al estudiante.
- Postcondición: Se genera un listado ordenado de vacantes recomendadas

CU05 – Notificaciones

- Actor: Estudiante / Empresa
- RF Asociado: RF11
- Precondición: Debe existir un evento que genere una notificación.
- Flujo principal:
 1. Se detecta un evento en cualquier microservicio (postulación, cambio de estado, nueva vacante, evaluación, recordatorio).
 2. El microservicio de Notificaciones procesa el evento.
 3. Se genera el mensaje correspondiente (panel, correo, SMS si aplica).
 4. La notificación se almacena en la base de datos.
 5. El usuario la recibe en su panel o correo electrónico.
- Postcondición: Notificación registrada, enviada y visible para el usuario

CU06 – Crear un convenio de práctica

- Actor: Empresa / Coordinador académico
- RF Asociado: RF12
- Precondición: Debe existir una postulación aceptada o estudiante asignado.
- Flujo principal:
 1. La empresa o el coordinador selecciona al estudiante en proceso de práctica.
 2. El sistema solicita información del convenio (fechas, tutor empresarial, tutor académico, modalidad).
 3. El microservicio de Prácticas valida la información y los roles del actor.
 4. El convenio se registra en la base de datos.
 5. El estudiante recibe notificación del convenio generado.
- Postcondición: Convenio creado y asociado al estudiante



CU07 – Evaluación final

- Actor: Empresa / Tutor Académico
- RF Asociado: RF13
- Precondición: Debe existir un convenio de práctica activo.
- Flujo principal:
 1. El tutor (empresarial o académico) accede al módulo de evaluación.
 2. Ingresa desempeño, calificaciones y observaciones del estudiante.
 3. El microservicio de Prácticas valida formato y permisos.
 4. Se guarda la evaluación en la base de datos.
 5. El estudiante recibe notificación del resultado.
- Postcondición: Evaluación final registrada en el sistema.

CU08 – Descarga de Métricas

- Actor: Coordinador Académico / Administrador Institucional
- RF Asociado: RF16
- Precondición: Debe existir datos históricos de vacantes, postulaciones o prácticas.
- Flujo principal:
 1. El actor selecciona el tipo de informe (programa, facultad, periodo, sede...).
 2. El microservicio de Analítica procesa la consulta.
 3. Genera archivo PDF/Excel con métricas (empleabilidad, tasa de práctica, postulaciones).
 4. El sistema pone disponible el archivo para descarga.
 5. El usuario descarga el informe.
- Postcondición: Reporte descargado con los indicadores actuales.

5. Fase de Investigación y Análisis

1. **Mapeo de Stakeholders:** Identificación de necesidades específicas mediante entrevistas a oficinas de prácticas y estudiantes que sufren la brecha de inserción laboral.
2. **Análisis de Mercado:** Estudio de las tasas de desempleo juvenil en Colombia y cómo la falta de canales especializados afecta la transición de academia a empresa.



3. **Benchmarking:** Evaluación de limitaciones en plataformas como LinkedIn o CompuTrabajo, las cuales mezclan ofertas para profesionales experimentados con las de practicantes, generando frustración en el estudiante.

6. DISEÑO DE LA ARQUITECTURA

6.1 Microservicios definidos

SERVICIO	LENGUAJE	FUNCIÓN
API Gateway	Nginx/Kong	Enrutamiento, seguridad
Auth Service	Node.js + JWT	Login y roles
Users Service	Node.js	Perfiles
Companies Service	Spring Boot	Empresas
Vancancies Service	Spring Boot	Vacantes
Matching Service	Python	Algoritmo
Applications Service	Node.js	Postulaciones
Practices Service	Spring Boot	Prácticas
Notifications Service	Node.js + RabbitMQ	Notificaciones
Analytics Service	Python	Métricas
Files Service	Node.js	Documentos (S3)
Audit Service	MongoDB + Node.js	Logs

6.2 Persistencia Distribuida - Database per Service

El modelo de persistencia distribuida asigna una base de datos independiente por microservicio, asegurando aislamiento, autonomía y escalabilidad. Esto se

fundamenta en Domain-Driven Design (DDD), donde cada contexto delimitado (bounded context) tiene su propio repositorio de información.

¿Por qué se escogió este enfoque?

- Garantiza independencia entre dominios funcionales
- Evita bloqueos y cuellos de botella típicos de bases centralizadas
- Incrementa la seguridad al separar datos sensibles (por ejemplo, logs vs perfiles estudiantiles)
- Permite replicación y escalado selectivo
- Facilita optimización según tipo de carga (lectura intensiva, escritura intensiva, consultas analíticas)
- Reduce la posibilidad de fallos catastróficos

Tecnologías seleccionadas

Microservicio	Tipo de dato	Motor	Justificación
Perfiles / Vacantes / Postulaciones	Relacional	PostgreSQL	Integridad referencial, ACID, rendimiento estable
Prácticas	Relacional	PostgreSQL	Modelos jerárquicos de convenios y evaluaciones
Notificaciones	NoSQL ligero	Redis	Colas rápidas, TTL de mensajes
Matching	Dataset dinámico	MongoDB	Flexibilidad para datos no estructurados
Auditoría	Documental	MongoDB	Consultas rápidas de logs

¿Qué aporta al proyecto?

- Seguridad granular
- Flexibilidad para crecimiento
- Protección ante fallos

- Soporte para analítica histórica
- Escalabilidad adaptada a cada microservicio

7. DESPLIEGUE EN LA NUBE

Tecnologías seleccionadas:

Kubernetes (EKS/AKS/GKE)

- Estándar actual para ejecutar microservicios
- Proporciona autoescalado horizontal (HPA)
- Ofrece reinicio automático de contenedores
- Permite actualizaciones sin detener el sistema (rolling updates)

¿Qué aporta al proyecto?

- Disponibilidad continua
- Resistencia a fallos
- Capacidad de crecer sin rediseñar la arquitectura

Docker

- Garantiza ambientes idénticos entre desarrollo, pruebas y producción.
- Simplifica la portabilidad del sistema entre nubes.

¿Qué aporta?

- Reducción de errores.
- Entornos reproducibles.

API Gateway + Load Balancer

- Centraliza autenticación, rate limiting y filtros de seguridad.
- Controla el tráfico hacia los microservicios.

¿Qué aporta?

- Seguridad consolidada.
- Visibilidad unificada del tráfico.



Storage S3 / Blob

- Escalable, económico y de baja latencia.

¿Qué aporta?

- Almacenamiento confiable de documentos de estudiantes y convenios.

Monitoreo con Prometheus & Grafana

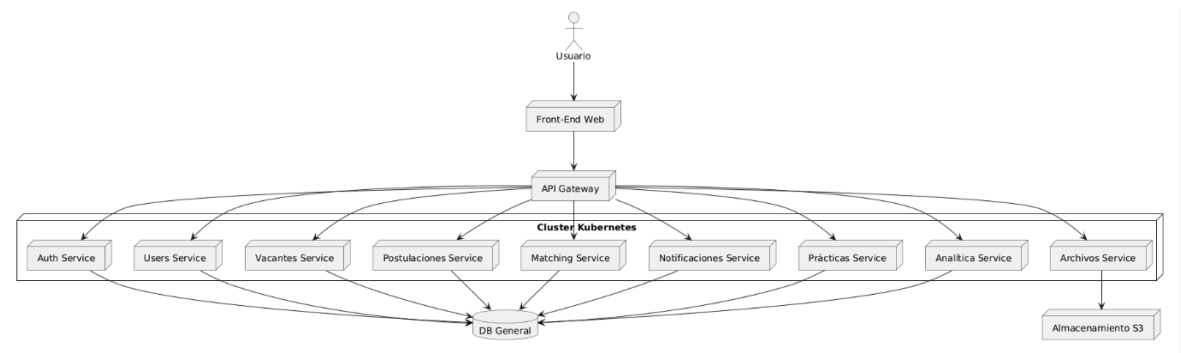
- Permite visualizar métricas por microservicio.
- Detecta comportamientos anómalos.

¿Qué aporta?

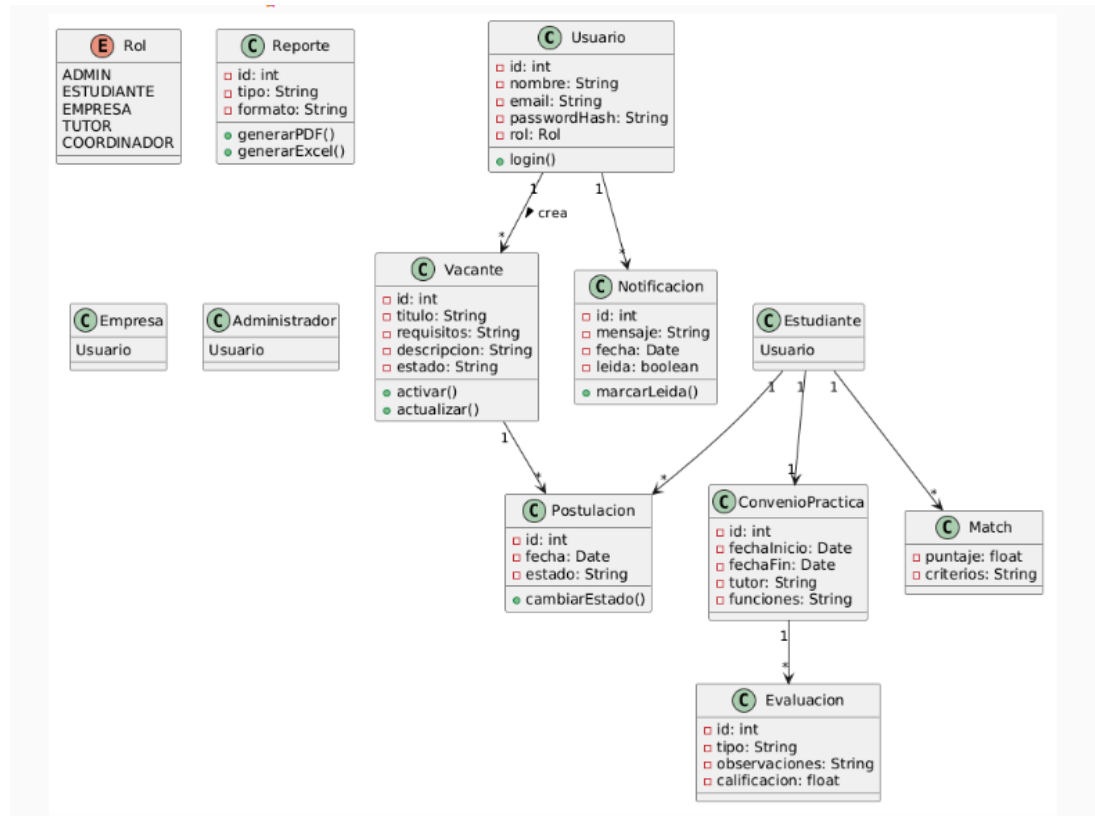
- Mantenimiento predictivo.
- Observabilidad completa.

8. Diagramas

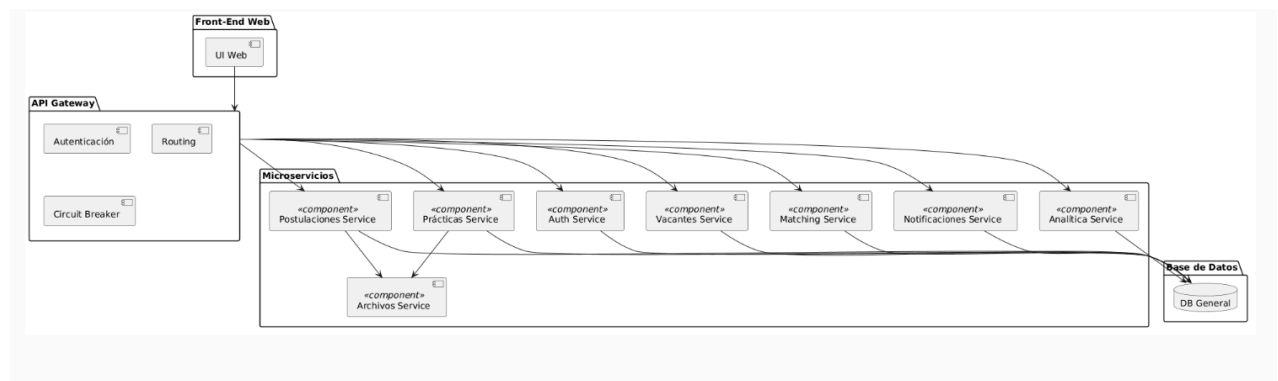
8.1 Diagrama de Despliegue



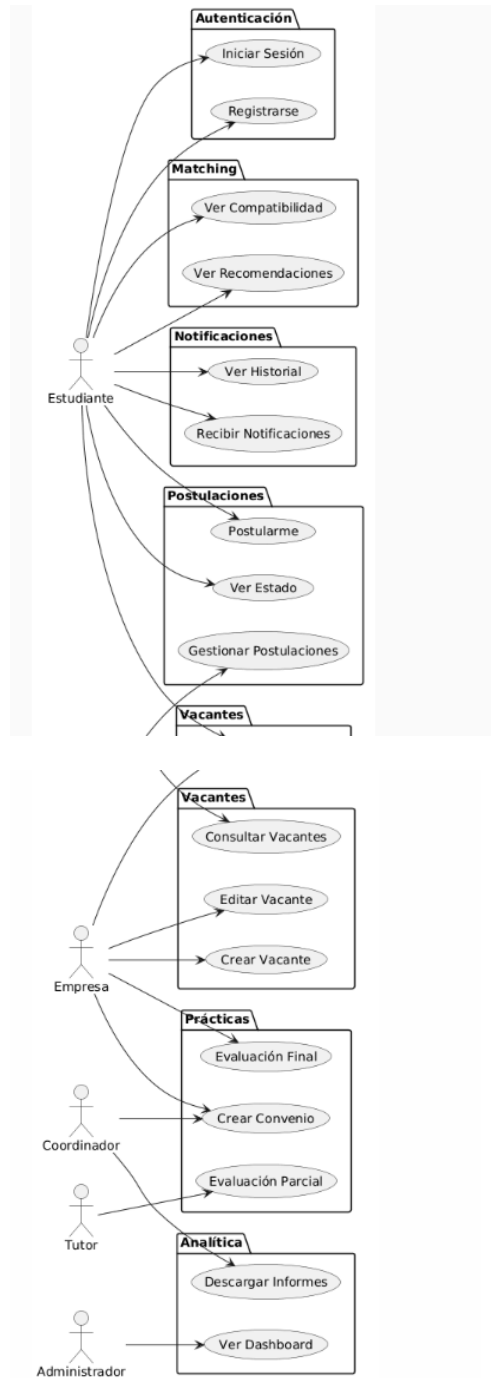
8.2 Diagrama de clases



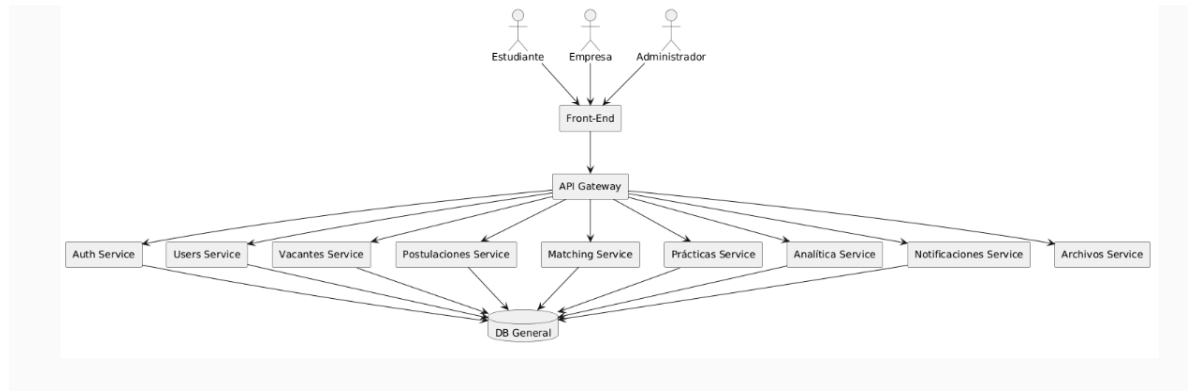
8.3 Diagrama de componentes



8.4 Diagrama de casos de uso



8.5 Diagrama UML General



9. Aseguramiento de la Calidad

9.1 Características ISO/IEC 25010

Aplicación en el proyecto:

Funcionalidad

- Pruebas de precisión del algoritmo de matching.

Fiabilidad

- Pruebas de carga sobre microservicios críticos.

Usabilidad

- Interfaz accesible bajo WCAG 2.1.

Rendimiento

- Microservicios capaces de responder en < 1.5 s.

Seguridad

- Validación JWT, cifrado y pruebas de penetración.

Mantenibilidad

- Código modular alineado con DDD.

Portabilidad



- Todo corre en contenedores Docker.

10. PROTOTIPO DE INFRAESTRUCTURA COMO CÓDIGO (Terraform)

10.1 Descripción General

Para el despliegue de la plataforma de empleabilidad se implementó un prototipo de Infraestructura como Código (IaC) utilizando Terraform, herramienta que permite definir, versionar y provisionar recursos en la nube mediante archivos de configuración declarativos. Este enfoque garantiza entornos reproducibles, elimina la configuración manual y permite gestionar la infraestructura junto al código fuente del proyecto bajo control de versiones.

Repositorio GitHub:

<https://github.com/Janka033/terraform.git>

10.2 Recursos Provisionados

El prototipo cubre tres recursos fundamentales alineados con la arquitectura de microservicios definida en la sección 6:

VPC (Red Privada Virtual)

Se configuró una VPC en la región us-east-1 con subredes públicas destinadas al API Gateway y Load Balancer, y subredes privadas para alojar los microservicios. Un NAT Gateway centralizado permite que los pods del clúster accedan a internet sin exponerse directamente, cumpliendo con los requisitos de seguridad RNF03 y RNF04.

Clúster EKS (Kubernetes en AWS)

Se provisiona un clúster EKS con Kubernetes 1.30 que orquesta los contenedores Docker de cada microservicio. El grupo de nodos inicia con 2 instancias t3.medium con capacidad de escalar hasta 6, habilitando el autoescalado horizontal (HPA) requerido en RNF08. El endpoint público permite la administración remota del clúster mediante kubectl.

Bucket S3



Se crea un bucket de almacenamiento para documentos de los usuarios (hojas de vida, convenios, certificados de práctica), correspondiente al RF20 y al Files Service definido en la arquitectura. El nombre incluye el ambiente (dev/prod) para separar entornos y evitar colisiones.

10.3 Tecnologías y Versiones

Componente	Versión
Terraform CLI	$\geq 1.5.0$
Provider AWS	5.0
Módulo VPC	5.8.1
Módulo EKS	20.8.5
Kubernetes	1.30

10.4 Alcance del Prototipo

Este prototipo corresponde al primer entregable e incluye únicamente la infraestructura base. Los siguientes recursos se incorporarán en entregas posteriores conforme avance el desarrollo de cada microservicio:

- Bases de datos RDS/PostgreSQL por microservicio (Database per Service)
- Instancias Redis para el servicio de notificaciones
- Clúster ElasticSearch para el buscador de vacantes
- Monitoreo con Prometheus y Grafana
- Configuración de dominios y certificados SSL

11. Historias de Usuario

MÓDULO DE REGISTRO Y AUTENTICACIÓN

HU01 – Registro como estudiante

Como estudiante,
quiero crear una cuenta en la plataforma,

para poder acceder a vacantes, postularme y recibir recomendaciones personalizadas.

Criterios de aceptación:

- El sistema valida que el correo no esté registrado.
- El usuario queda creado en el microservicio de usuarios.
- Se genera token JWT para el inicio de sesión.

HU02 – Registro como empresa

Como estudiante,
quiero crear una cuenta en la plataforma,
para poder acceder a vacantes, postularme y recibir recomendaciones personalizadas.

Criterios de aceptación:

- El sistema valida que el correo no esté registrado.
- El usuario queda creado en el microservicio de usuarios.
- Se genera token JWT para el inicio de sesión.

HU03 – Iniciar Sesión

Como usuario registrado,
quiero acceder a mi cuenta con mis credenciales,
para ver mi panel y utilizar todas las funcionalidades permitidas por mi rol.

Criterios de aceptación:

- Validación de credenciales en el microservicio de autenticación.
- Respuesta con token JWT.

MÓDULO DE VACANTES

HU04 – Crear una vacante

Como empresa,
quiero registrar una nueva vacante,
para atraer candidatos adecuados a mi oferta.

Criterios de aceptación:

- Se validan campos obligatorios (título, requisitos, descripción).
- El sistema almacena la vacante.
- La vacante aparece como “Activa”.

HU05 – Editar vacante

Como empresa,
quiero modificar los datos de una vacante,
para actualizar requisitos o corregir información.

Criterios de aceptación:

- Solo puede editar vacantes propias.
- El sistema registra la actualización como nueva versión.

HU06 – Consultar vacantes

Como estudiante,
quiero navegar y filtrar vacantes por carrera, modalidad, salario o ubicación,
para encontrar oportunidades compatibles con mi perfil.

Criterios de aceptación:

- El sistema muestra filtros avanzados.
- Responde rápidamente (<1.5 s).

MÓDULO DE POSTULACIONES

HU07 – Postularme a una vacante

Como estudiante,
quiero enviar mi hoja de vida y documentos a una vacante,
para que la empresa me tenga en cuenta en su proceso.

Criterios de aceptación:

- Los documentos se suben al microservicio de Archivos.
- La postulación se registra correctamente.
- Se envía notificación automática.

HU08 – Ver estado de mi postulación

Como estudiante,

quiero ver el estado actual de mis postulaciones (Enviada, Revisada, Seleccionado, Rechazado),
para hacer seguimiento a mis solicitudes.

Criterios de aceptación:

- Se muestra el historial de cambios.
- Información actualizada en tiempo real.

HU09 – Gestionar postulaciones recibidas

Como empresa,
quiero ver todos los candidatos que postularon a una vacante,
para revisar sus perfiles y tomar decisiones.

Criterios de aceptación:

- Se listan postulaciones ordenadas por fecha.
- Se pueden ver documentos adjuntos.

MÓDULO DE MATCHING INTELIGENTE

HU10 – Recibir recomendaciones automáticas

Como estudiante,
quiero que el sistema me recomiende vacantes acordes a mi perfil académico,
para encontrar oportunidades relevantes sin buscarlas manualmente.

Criterios de aceptación:

- El sistema analiza mi perfil.
- El microservicio de Matching genera ranking.
- Las recomendaciones se muestran en el panel.

HU11 – Ver puntaje de compatibilidad

Como estudiante,
quiero visualizar el porcentaje de compatibilidad entre mi perfil y una vacante,
para priorizar oportunidades adecuadas.

Criterios de aceptación:

- El sistema muestra el puntaje calculado.

- Se explica brevemente el criterio (skills, estudio, requisitos).

MÓDULO DE NOTIFICACIONES

HU12 – Recibir notificaciones de eventos

Como usuario,
quiero recibir notificaciones cuando haya cambios importantes (nueva vacante, postulación, evaluación),
para estar informado del proceso.

Criterios de aceptación:

- El microservicio de Notificaciones registra y envía eventos.
- Deben mostrarse en el panel.
- Enviar correo opcionalmente.

HU13 – Ver historial de notificaciones

Como usuario,
quiero visualizar el historial completo de mis notificaciones,
para no perder información importante.

Criterios de aceptación:

- Se listan notificaciones por fecha.
- El usuario puede marcarlas como leídas.

MÓDULO DE PRÁCTICAS

HU14 – Crear convenio de práctica

Como empresa o coordinador académico,
quiero registrar el convenio de práctica para un estudiante,
para formalizar el proceso y dar inicio a la práctica.

Criterios de aceptación:

- Se ingresan fechas, tutor, funciones.
- El microservicio de Prácticas guarda el convenio.

HU15 – Registrar evaluaciones parciales

Como tutor académico,



quiero agregar evaluaciones periódicas del desempeño del estudiante, para hacer seguimiento de la práctica.

Criterios de aceptación:

- Se registran observaciones y calificaciones.
- Se guarda historial evaluativo.

HU16 – Registrar evaluación final

Como empresa,
quiero evaluar al estudiante al finalizar la práctica,
para generar la calificación final requerida institucionalmente.

Criterios de aceptación:

- Se llena el formulario final.
- El sistema notifica al estudiante.

MÓDULO DE MÉTRICAS

HU17 – Descarga de informes

Como coordinador académico,
quiero descargar informes de empleabilidad, prácticas y postulaciones,
para analizarlos en procesos de mejora o acreditación.

Criterios de aceptación:

- Los informes se generan en PDF/Excel.
- El microservicio de Analítica procesa la solicitud.
- Archivo disponible para descarga.

HU18 – Ver métricas en panel

Como administrador,
quiero ver un dashboard con indicadores clave,
para monitorear el desempeño institucional.

Criterios de aceptación:

- Gráficos de vacantes, postulaciones, prácticas, tasas de empleabilidad.

- Información actualizada periódicamente.

12. Patrones de Diseño

API Gateway Pattern: Controla autenticación, seguridad y enruta peticiones.

Repository Pattern: Permite independencia entre lógica y persistencia.

Factory Pattern: Crea distintos tipos de usuario sin duplicación.

Observer Pattern: Genera notificaciones ante cambios de estado.

Circuit Breaker: Evita fallos en cascada.

Saga Pattern: Administra procesos largos como convenios y evaluaciones.

DTO Pattern: Evita exponer la estructura interna de los servicios

